Міністерство освіти і науки України

КПІ ім. Ігоря Сікорського

Факультет інформатики та обчислювальної техніки

Кафедра інформаційних систем та технологій

Лабораторна робота № 4

З дисципліни “Сучасні технології розробки WEB-застосувань на платформі Microsoft.Net ”

студентки ІІІ курсу ФІОТ

групи ІК-12

Макарчук Ольги

Перевірив:
Бардін В.

Київ 2023

Мета:

1. Ознайомитися з основами створення REST веб-API та методологією C4 для відображення архітектури системи.
2. Ознайомитися з основами створення ER-діаграм для представлення структури бази даних.

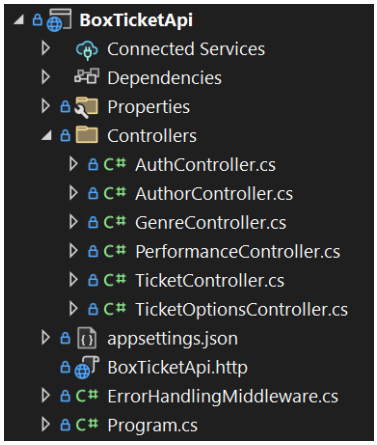
Завдання:

1. Використовуючи архітектуру розроблену в попередній роботі Імплементувати REST веб-API, використавши N Layered архітектуру.
2. Покрити модульними та інтеграційними тестами основні компоненти рішення.
3. Експортувати розроблене API до Postman.

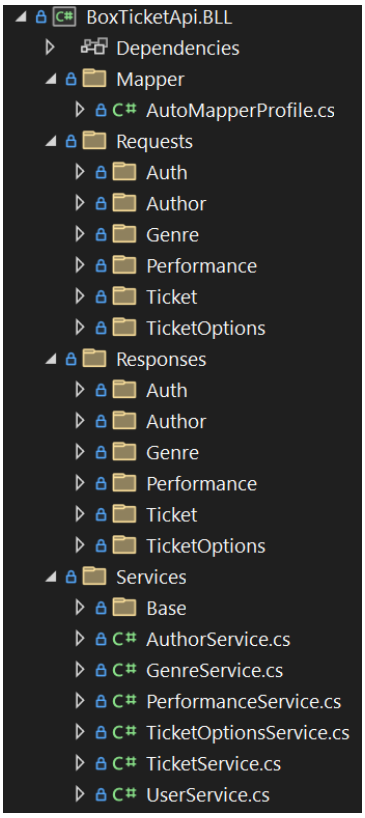
Варіант 3:

3	Театральна каса. Розповсюдження квитків	1. Афіша вистав надає функції пошуку потрібної вистави за автором, назвою, жанром та датою. 2. Кожній виставі відповідає кількість квитків різної вартості. 3. Квитки можна безпосередньо продати чи попередньо забронювати та згодом перевести їх у стан проданих. **Функціональні вимоги:** 1. Ведення афіши вистав; 2. Продаж квитків

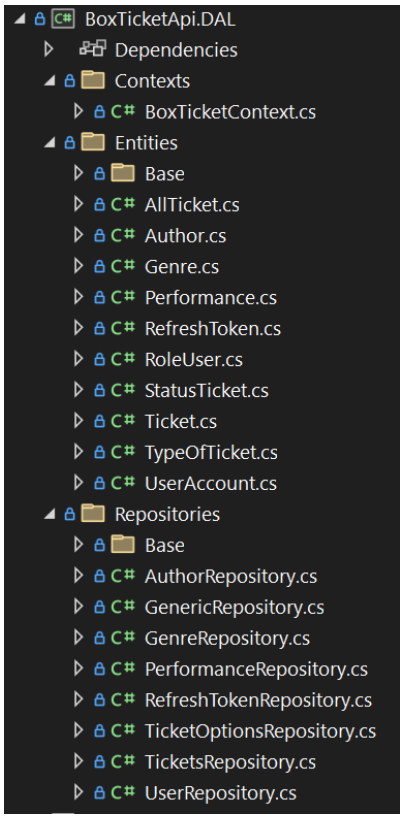
Структура WebApi:



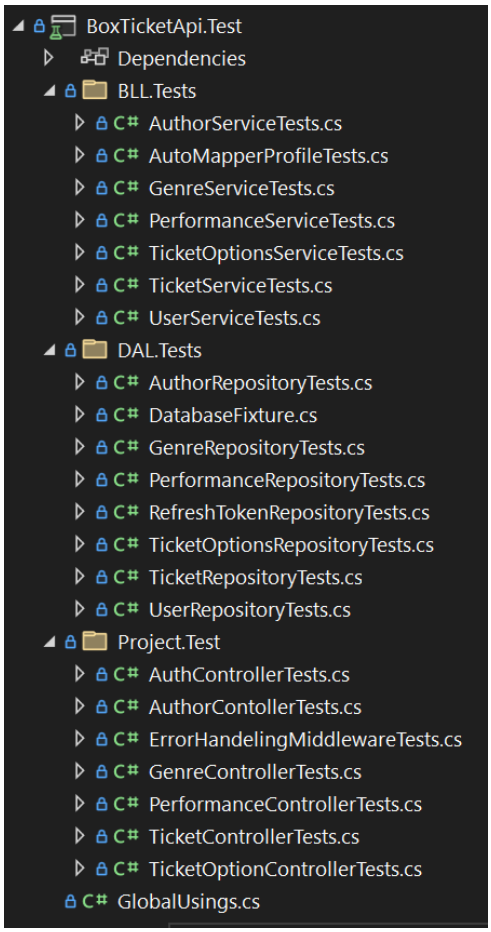
Структура BLL:



Структура DAL:



Структура Unit тестів:



Кінцеві точки REST API(Endpoints)

Unit Тести:

```
using AutoMapper;
using BoxTicketApi.BLL.Requests.Author;
using BoxTicketApi.BLL.Responses.Author;
using BoxTicketApi.BLL.Services;
using BoxTicketApi.DAL.Entities;
using BoxTicketApi.DAL.Repositories.Base;
using Moq;

namespace BoxTicketApi.Test.BLL.Tests
{
    public class AuthorServiceTests
    {
        private readonly Mock<IMapper> _mapperMock = new();
        private readonly Mock<IAuthorRepository> _authorRepositoryMock = new();
        private AuthorService _authorService = null!;

        [Fact]
        public async Task AddAuthor_ShouldReturnAuthor()
        {
            var author = new Author
            {
                Id = 1,
                FirstName = "Name1",
                LastName = "Last1",
                MiddleName = "Middle1",
            };

            var request = new AuthorRequest
            {
                FirstName = "Name1",
                LastName = "Last1",
                MiddleName = "Middle1",
            };

            var expectedAuthor = new AuthorResponse
            {
                Id = 1,
                FirstName = "Name1",
                LastName = "Last1",
                MiddleName = "Middle1",
            };

            _mapperMock.Setup(m => m.Map<Author>(request))
                .Returns(author);
            _mapperMock.Setup(m => m.Map<AuthorResponse>(It.IsAny<Author>()))
                .Returns(expectedAuthor);
            _authorService = new AuthorService(_authorRepositoryMock.Object,
_mapperMock.Object);

            // Act
            var adderAuthor = await _authorService.AddAuthor(request);

            // Assert
            Assert.NotNull(adderAuthor);
            Assert.Equal(request.LastName, adderAuthor.LastName);
        }

        [Fact]
        public async Task GetAllAuthor_ShouldReturnAuthors()
        {
            List<Author> authors = new()
            {
                new Author{Id = 1, FirstName = "Name1", LastName = "Last1",
MiddleName = "Middle1"},
                new Author{Id = 2, FirstName = "Name2", LastName = "Last2",
MiddleName = "Middle2"},
                new Author{Id = 3, FirstName = "Name3", LastName = "Last3",
MiddleName = "Middle3"},
```

```
            };

            List<AuthorResponse> authorsExpecter = new()
            {
                new AuthorResponse{Id = 1, FirstName = "Name1", LastName = "Last1",
MiddleName = "Middle1"},
                new AuthorResponse{Id = 2, FirstName = "Name2", LastName = "Last2",
MiddleName = "Middle2"},
                new AuthorResponse{Id = 3, FirstName = "Name3", LastName = "Last3",
MiddleName = "Middle3"},
            };
            var author = new Author
            {
                Id = 1,
                FirstName = "John",
                LastName = "Doe",
                MiddleName = "Smith",
            };

            _mapperMock.Setup(m => m.Map<List<AuthorResponse>>(It.IsAny<Author>()))
                .Returns(authorsExpecter);
            _authorService = new AuthorService(_authorRepositoryMock.Object,
_mapperMock.Object);

            // Act
            var allAuthors = await _authorService.GetAllAuthor();

            // Assert
            Assert.NotNull(allAuthors);
            Assert.Equal(3, allAuthors.Count);
        }
    }
}


using AutoMapper;
using BoxTicketApi.BLL.Mapper;
using BoxTicketApi.BLL.Requests.Genre;
using BoxTicketApi.DAL.Entities;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace BoxTicketApi.Test.BLL.Tests
{
    public class AutoMapperProfileTests
    {
        private readonly IMapper _mapper;

        public AutoMapperProfileTests()
        {
            var mapperConfig = new MapperConfiguration(cfg =>
            {
                cfg.AddProfile(new AutoMapperProfile());
            });

            _mapper = mapperConfig.CreateMapper();
        }

        [Fact]
        public void Map_GenreRequest_To_Genre()
        {
            var genreRequest = new GenreRequest();

            var mappedGenre = _mapper.Map<GenreRequest, Genre>(genreRequest);

            Assert.NotNull(mappedGenre);
        }
```

```
    }
}


using AutoMapper;
using BoxTicketApi.BLL.Requests.Author;
using BoxTicketApi.BLL.Requests.Genre;
using BoxTicketApi.BLL.Responses.Author;
using BoxTicketApi.BLL.Responses.Genre;
using BoxTicketApi.BLL.Services;
using BoxTicketApi.DAL.Entities;
using BoxTicketApi.DAL.Repositories.Base;
using Moq;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace BoxTicketApi.Test.BLL.Tests
{
    public class GenreServiceTests
    {
        private readonly Mock<IMapper> _mapperMock = new();
        private readonly Mock<IGenreRepository> _genreRepositoryMock = new();
        private GenreService _genreService = null!;

        [Fact]
        public async Task AddGenreShouldReturnGenre()
        {
            var genre = new Genre
            {
                Id = 1,
                NameGenre = "Genre1"
            };

            var request = new GenreRequest
            {
                NameGenre = "Genre1"
            };

            var expectedGenre = new GenreResponse
            {
                Id = 1,
                NameGenre = "Genre1"
            };

            _mapperMock.Setup(m => m.Map<Genre>(request))
                .Returns(genre);
            _mapperMock.Setup(m => m.Map<GenreResponse>(It.IsAny<Genre>()))
                .Returns(expectedGenre);
            _genreService = new GenreService(_genreRepositoryMock.Object,
_mapperMock.Object);

            // Act
            var adderGenre = await _genreService.AddGenre(request);

            // Assert
            Assert.NotNull(adderGenre);
            Assert.Equal(request.NameGenre, adderGenre.NameGenre);
        }

        [Fact]
        public async Task GetAllGenre_ShouldReturnGenres()
        {
            List<Genre> genres = new()
            {
                new Genre(){Id=1, NameGenre="Genre1"},
                new Genre(){Id=2, NameGenre="Genre1"},
                new Genre(){Id=3, NameGenre="Genre1"}
```

```
            };

            List<GenreResponse> genresExpected = new()
            {
                new GenreResponse(){Id=1, NameGenre="Genre1"},
                new GenreResponse(){Id=2, NameGenre="Genre1"},
                new GenreResponse(){Id=3, NameGenre="Genre1"}
            };

            _mapperMock.Setup(m => m.Map<List<GenreResponse>>(It.IsAny<Author>()))
                .Returns(genresExpected);
            _genreService = new GenreService(_genreRepositoryMock.Object,
_mapperMock.Object);

            // Act
            var allAuthors = await _genreService.GetAllGenre();

            // Assert
            Assert.NotNull(allAuthors);
            Assert.Equal(3, allAuthors.Count);
        }
    }
}


using AutoMapper;
using BoxTicketApi.BLL.Mapper;
using BoxTicketApi.BLL.Requests.Genre;
using BoxTicketApi.DAL.Entities;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace BoxTicketApi.Test.BLL.Tests
{
    public class AutoMapperProfileTests
    {
        private readonly IMapper _mapper;

        public AutoMapperProfileTests()
        {
            var mapperConfig = new MapperConfiguration(cfg =>
            {
                cfg.AddProfile(new AutoMapperProfile());
            });

            _mapper = mapperConfig.CreateMapper();
        }

        [Fact]
        public void Map_GenreRequest_To_Genre()
        {
            var genreRequest = new GenreRequest();

            var mappedGenre = _mapper.Map<GenreRequest, Genre>(genreRequest);

            Assert.NotNull(mappedGenre);
        }
    }
}
using AutoMapper;
using BoxTicketApi.BLL.Requests.Author;
using BoxTicketApi.BLL.Requests.Genre;
using BoxTicketApi.BLL.Responses.Author;
using BoxTicketApi.BLL.Responses.Genre;
using BoxTicketApi.BLL.Services;
using BoxTicketApi.DAL.Entities;
using BoxTicketApi.DAL.Repositories.Base;
```

```
using Moq;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace BoxTicketApi.Test.BLL.Tests
{
    public class GenreServiceTests
    {
        private readonly Mock<IMapper> _mapperMock = new();
        private readonly Mock<IGenreRepository> _genreRepositoryMock = new();
        private GenreService _genreService = null!;

        [Fact]
        public async Task AddGenreShouldReturnGenre()
        {
            var genre = new Genre
            {
                Id = 1,
                NameGenre = "Genre1"
            };

            var request = new GenreRequest
            {
                NameGenre = "Genre1"
            };

            var expectedGenre = new GenreResponse
            {
                Id = 1,
                NameGenre = "Genre1"
            };

            _mapperMock.Setup(m => m.Map<Genre>(request))
                .Returns(genre);
            _mapperMock.Setup(m => m.Map<GenreResponse>(It.IsAny<Genre>()))
                .Returns(expectedGenre);
            _genreService = new GenreService(_genreRepositoryMock.Object,
_mapperMock.Object);

            // Act
            var adderGenre = await _genreService.AddGenre(request);

            // Assert
            Assert.NotNull(adderGenre);
            Assert.Equal(request.NameGenre, adderGenre.NameGenre);
        }

        [Fact]
        public async Task GetAllGenre_ShouldReturnGenres()
        {
            List<Genre> genres = new()
            {
                new Genre(){Id=1, NameGenre="Genre1"},
                new Genre(){Id=2, NameGenre="Genre1"},
                new Genre(){Id=3, NameGenre="Genre1"}
            };

            List<GenreResponse> genresExpected = new()
            {
                new GenreResponse(){Id=1, NameGenre="Genre1"},
                new GenreResponse(){Id=2, NameGenre="Genre1"},
                new GenreResponse(){Id=3, NameGenre="Genre1"}
            };

            _mapperMock.Setup(m => m.Map<List<GenreResponse>>(It.IsAny<Author>()))
                .Returns(genresExpected);
```

```
            _genreService = new GenreService(_genreRepositoryMock.Object,
_mapperMock.Object);

            // Act
            var allAuthors = await _genreService.GetAllGenre();

            // Assert
            Assert.NotNull(allAuthors);
            Assert.Equal(3, allAuthors.Count);
        }
    }
}
```

```
using AutoMapper;
using BoxTicketApi.BLL.Requests.Genre;
using BoxTicketApi.BLL.Requests.Performance;
using BoxTicketApi.BLL.Responses.Genre;
using BoxTicketApi.BLL.Responses.Performance;
using BoxTicketApi.BLL.Services;
using BoxTicketApi.DAL.Entities;
using BoxTicketApi.DAL.Repositories.Base;
using Moq;
using Newtonsoft.Json.Linq;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace BoxTicketApi.Test.BLL.Tests
{
    public class PerformanceServiceTests
    {
        private readonly Mock<IMapper> _mapperMock = new();
        private readonly Mock<IGenreRepository> _genreRepositoryMock = new();
        private readonly Mock<IPerformanceRepository> _performanceRepositoryMock
= new();
        private readonly Mock<IAuthorRepository> _authorRepositoryMock = new();
        private PerformanceService _performanceService = null!;
```

```
public Genre genre = new Genre() { Id = 1, NameGenre = "Genre1" };
public Author author = new Author() { Id = 1, FirstName = "Test", MiddleName = "Test", LastName = "Test" };

public Performance performance = new Performance() { IdAuthor = 1, IdGenre = 1, DateTimeEvent = DateTime.Now, PerformanceName = "Performance1" };

public PerformanceRequest request = new PerformanceRequest() { IdAuthor = 1, IdGenre = 1, DateTimeEvent = DateTime.Now, PerformanceName = "Performance1" };

public PerformanceResponse expectedPerformance = new PerformanceResponse() { IdAuthor = 1, IdGenre = 1, DateTimeEvent = DateTime.Now, PerformanceName = "Performance1", IdPerformance = 1 };

public List<PerformanceResponse> expectedPerformances = new List<PerformanceResponse>();
[Fact]
public async Task AddPerformance_ShouldReturnPerformance()
{
    _mapperMock.Setup(m => m.Map<Performance>(request))
        .Returns(performance);
    _mapperMock.Setup(m => m.Map<PerformanceResponse>(It.IsAny<Performance>()))
        .Returns(expectedPerformance);

    _performanceService = new PerformanceService(_mapperMock.Object, _performanceRepositoryMock.Object, _authorRepositoryMock.Object, _genreRepositoryMock.Object);
    _authorRepositoryMock.Setup(repo => repo.GetByIdAsync(It.IsAny<int>()))
```

```
        .ReturnsAsync(author);
        _genreRepositoryMock.Setup(repo => repo.GetByIdAsync(It.IsAny<int>()))
        .ReturnsAsync(genre);

        var adderPerformance = await _performanceService.AddPerformance(request);

        Assert.NotNull(adderPerformance);
        Assert.Equal(request.PerformanceName, adderPerformance.PerformanceName);
    }

    [Fact]
    public async Task AddPerformance_ShouldReturnExeptionNotFoundGenre()
    {
        _mapperMock.Setup(m => m.Map<Performance>(request))
            .Returns(performance);
        _mapperMock.Setup(m => m.Map<PerformanceResponse>(It.IsAny<Performance>()))
            .Returns(expectedPerformance);

        _performanceService = new PerformanceService(_mapperMock.Object, _performanceRepositoryMock.Object, _authorRepositoryMock.Object, _genreRepositoryMock.Object);
        _authorRepositoryMock.Setup(repo => repo.GetByIdAsync(It.IsAny<int>()))
        .ReturnsAsync(author);

        var exception = await Assert.ThrowsAsync<Exception>(async () => await _performanceService.AddPerformance(request));
```

```
        Assert.Equal($"Genre with id {request.IdAuthor} not found", exception.Message);
    }

    [Fact]
    public async Task AddPerformance_ShouldReturnExeptionNotFoundAuthor()
    {
        _mapperMock.Setup(m => m.Map<Performance>(request))
            .Returns(performance);
        _mapperMock.Setup(m => m.Map<PerformanceResponse>(It.IsAny<Performance>()))
            .Returns(expectedPerformance);

        _performanceService = new PerformanceService(_mapperMock.Object, _performanceRepositoryMock.Object, _authorRepositoryMock.Object, _genreRepositoryMock.Object);

        _genreRepositoryMock.Setup(repo => repo.GetByIdAsync(It.IsAny<int>()))
        .ReturnsAsync(genre);

        var exception = await Assert.ThrowsAsync<Exception>(async () => await _performanceService.AddPerformance(request));

        Assert.Equal($"Author with id {request.IdAuthor} not found", exception.Message);
    }

    [Fact]
    public async Task GetAllPerformances_ShouldReturnAllperformances()
    {
        expectedPerformances.Add(expectedPerformance);
```

```
        _mapperMock.Setup(m => m.Map<List<PerformanceResponse>>(It.IsAny<Performance>()))
            .Returns(expectedPerformances);
        _performanceService = new PerformanceService(_mapperMock.Object, _performanceRepositoryMock.Object, _authorRepositoryMock.Object, _genreRepositoryMock.Object);

        var performances = await _performanceService.GetAllPerformances();

        Assert.NotNull(performances);
        Assert.Single(performances);
    }

    [Fact]
    public async Task GetPerformancesByAuthor_ShouldReturnPerformances()
    {
        List<Performance> performancesList = new List<Performance>();
        performancesList.Add(performance);
        var requestByAuthor = new PerformancesByAuthorRequest() { idAuthor = 1 };
        expectedPerformances.Add(expectedPerformance);
        _mapperMock.Setup(m => m.Map<List<PerformanceResponse>>(It.IsAny<List<Performance>>()))
            .Returns(expectedPerformances);
        _performanceService = new PerformanceService(_mapperMock.Object, _performanceRepositoryMock.Object, _authorRepositoryMock.Object, _genreRepositoryMock.Object);

        _performanceRepositoryMock.Setup(repo => repo.GetPerformancesByAuthor(requestByAuthor.idAuthor))
            .ReturnsAsync(performancesList);
```

```
        var performances = await _performanceService.GetPerformancesByAuthor(requestByAuthor);

        Assert.NotNull(performances);
        Assert.Single(performances);
    }

    [Fact]
    public async Task GetPerformancesByAuthor_ShouldReturnExeption()
    {
        List<Performance> performancesList = new List<Performance>();
        var requestByAuthor = new PerformancesByAuthorRequest() { idAuthor = 1 };
        expectedPerformances.Add(expectedPerformance);
        _mapperMock.Setup(m => m.Map<List<PerformanceResponse>>(It.IsAny<List<Performance>>()))
            .Returns(expectedPerformances);
        _performanceService = new PerformanceService(_mapperMock.Object, _performanceRepositoryMock.Object, _authorRepositoryMock.Object, _genreRepositoryMock.Object);
        _performanceRepositoryMock.Setup(repo => repo.GetPerformancesByAuthor(requestByAuthor.idAuthor))
            .ReturnsAsync(performancesList);

        var exception = await Assert.ThrowsAsync<Exception>(async () => await _performanceService.GetPerformancesByAuthor(requestByAuthor));

        Assert.Equal($"Performances with author id {requestByAuthor.idAuthor} not found", exception.Message);
    }
```

```
    [Fact]
    public async Task GetPerformancesByDate_WhenValidDate_ShouldReturnPerformances()
    {
        List<Performance> performancesList = new List<Performance>();
        performancesList.Add(performance);
        var requestByDate = new PerformancesByDateRequest();
        requestByDate.Date = "2023-02-02";
        expectedPerformances.Add(expectedPerformance);
        _mapperMock.Setup(m => m.Map<List<PerformanceResponse>>(It.IsAny<List<Performance>>()))
            .Returns(expectedPerformances);
        _performanceService = new PerformanceService(_mapperMock.Object, _performanceRepositoryMock.Object, _authorRepositoryMock.Object, _genreRepositoryMock.Object);

        _performanceRepositoryMock.Setup(repo => repo.GetPerformancesByDate(requestByDate.dateTime))
            .ReturnsAsync(performancesList);

        var performances = await _performanceService.GetPerformancesByDate(requestByDate);

        Assert.NotNull(performances);
        Assert.Single(performances);
    }

    [Fact]
    public void Date_Setter_ShouldSetDateTime()
    {
```

```
        // Arrange
        var dateTime = new DateTime(2023, 12, 10);
        var requestByDate = new PerformancesByDateRequest();



        // Assert
        var exception = Assert.Throws<Exception>(() => requestByDate.Date = "2023-13-13");
        Assert.Equal("Неправильний формат дати. Використовуйте формат 'рік-день-місяць'.", exception.Message);
    }

    [Fact]
    public async Task GetPerformancesByDate_ShouldReturnExeption()
    {
        List<Performance> performancesList = new List<Performance>();
        var requestByDate = new PerformancesByDateRequest() { dateTime = new DateTime(2023, 12, 1) };
        expectedPerformances.Add(expectedPerformance);
        _mapperMock.Setup(m => m.Map<List<PerformanceResponse>>(It.IsAny<List<Performance>>()))
            .Returns(expectedPerformances);
        _performanceService = new PerformanceService(_mapperMock.Object, _performanceRepositoryMock.Object, _authorRepositoryMock.Object, _genreRepositoryMock.Object);

        _performanceRepositoryMock.Setup(repo => repo.GetPerformancesByDate(requestByDate.dateTime))
            .ReturnsAsync(performancesList);
```

```
        var exception = await Assert.ThrowsAsync<Exception>(async () => await
_performanceService.GetPerformancesByDate(requestByDate));

        Assert.Equal($"Performances with date {requestByDate.dateTime} not
found", exception.Message);
    }

    [Fact]
    public async Task GetPerformancesByGenre_ShouldReturnPerformances()
    {
        List<Performance> performancesList = new List<Performance>();
        performancesList.Add(performance);
        var requestByGenre = new PerformancesByGenreRequest() { idGenre=1};
        expectedPerformances.Add(expectedPerformance);
        _mapperMock.Setup(m                                                    =>
m.Map<List<PerformanceResponse>>(It.IsAny<List<Performance>>()))
            .Returns(expectedPerformances);
        _performanceService = new PerformanceService(_mapperMock.Object,
_performanceRepositoryMock.Object, _authorRepositoryMock.Object,
_genreRepositoryMock.Object);

        _performanceRepositoryMock.Setup(repo                                  =>
repo.GetPerformancesByGenre(requestByGenre.idGenre))
            .ReturnsAsync(performancesList);

        var performances = await
_performanceService.GetPerformancesByGenre(requestByGenre);

        Assert.NotNull(performances);
        Assert.Single(performances);
    }
```

```
    [Fact]
    public async Task GetPerformancesByGenre_ShouldReturnExeption()
    {
        List<Performance> performancesList = new List<Performance>();
        var requestByGenre = new PerformancesByGenreRequest() { idGenre = 1 };
        expectedPerformances.Add(expectedPerformance);
        _mapperMock.Setup(m => m.Map<List<PerformanceResponse>>(It.IsAny<List<Performance>>()))
            .Returns(expectedPerformances);
        _performanceService = new PerformanceService(_mapperMock.Object, _performanceRepositoryMock.Object, _authorRepositoryMock.Object, _genreRepositoryMock.Object);

        _performanceRepositoryMock.Setup(repo => repo.GetPerformancesByGenre(requestByGenre.idGenre))
            .ReturnsAsync(performancesList);

        var exception = await Assert.ThrowsAsync<Exception>(async () => await _performanceService.GetPerformancesByGenre(requestByGenre));

        Assert.Equal($"Performances with genre id {requestByGenre.idGenre} not found", exception.Message);
    }

    [Fact]
    public async Task GetPerformancesByName_ShouldReturnPerformances()
    {
        List<Performance> performancesList = new List<Performance>();
        performancesList.Add(performance);
```

```
        var requestByName = new PerformancesByNameRequest() { Name="name" };
        expectedPerformances.Add(expectedPerformance);
        _mapperMock.Setup(m => m.Map<List<PerformanceResponse>>(It.IsAny<List<Performance>>()))
            .Returns(expectedPerformances);
        _performanceService = new PerformanceService(_mapperMock.Object, _performanceRepositoryMock.Object, _authorRepositoryMock.Object, _genreRepositoryMock.Object);

        _performanceRepositoryMock.Setup(repo => repo.GetPerformancesByName(requestByName.Name))
            .ReturnsAsync(performancesList);

        var performances = await _performanceService.GetPerformancesByName(requestByName);

        Assert.NotNull(performances);
        Assert.Single(performances);
    }

    [Fact]
    public async Task GetPerformancesByName_ShouldReturnExeption()
    {
        List<Performance> performancesList = new List<Performance>();
        var requestByName = new PerformancesByNameRequest() { Name = "name" };
        expectedPerformances.Add(expectedPerformance);
        _mapperMock.Setup(m => m.Map<List<PerformanceResponse>>(It.IsAny<List<Performance>>()))
            .Returns(expectedPerformances);
```

_performanceService = new PerformanceService(_mapperMock.Object, _performanceRepositoryMock.Object, _authorRepositoryMock.Object, _genreRepositoryMock.Object);

_performanceRepositoryMock.Setup(repo => repo.GetPerformancesByName(requestByName.Name))
.ReturnsAsync(performancesList);

var exception = await Assert.ThrowsAsync<Exception>(async () => await _performanceService.GetPerformancesByName(requestByName));

Assert.Equal($"Performances with name {requestByName.Name} not found", exception.Message);
}
}
}

```
using AutoMapper;
using BoxTicketApi.BLL.Responses.TicketOptions;
using BoxTicketApi.BLL.Services;
using BoxTicketApi.DAL.Repositories.Base;
using Moq;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using BoxTicketApi.BLL.Requests.TicketOptions;
using BoxTicketApi.DAL.Entities;
using BoxTicketApi.BLL.Mapper;

namespace BoxTicketApi.Test.BLL.Tests
{
    public class TicketOptionsServiceTests
    {
        private readonly Mock<IMapper> _mapperMock = new();
        private readonly Mock<ITicketOptionsRepository> _ticketOptionsRepositoryMock = new();
        private TicketOptionsService _ticketOptionsService = null!;

        [Fact]
        public async Task GetAllAvailableTickets_WhenTicketsFound_ShouldReturnsTicketOptions()
        {
            var request = new GetOptionsRequest { IdPerformance = 1 };
            var allTickets = new List<AllTicket>
            {
                new AllTicket { IdType = 1, CoutOfTickets = 10 }
            };

            var purchasedSeats = new List<int> { 1, 3, 5 };
```

```
            _ticketOptionsRepositoryMock.Setup(repo =>
repo.GetAllTickets(request.IdPerformance))
                .ReturnsAsync(allTickets);

            _ticketOptionsRepositoryMock.Setup(repo =>
repo.GetBoughtSeatsByType(request.IdPerformance, It.IsAny<int>()))
                .ReturnsAsync((int idPerformance, int idType) =>
                {
                    return idType == 1 ? purchasedSeats : new List<int>();
                });

            _mapperMock.Setup(mapper =>
mapper.Map<OptionsResponse>(It.IsAny<AllTicket>()))
                .Returns((AllTicket ticketOptions) =>
                {
                    return new OptionsResponse
                    {
                        Id = ticketOptions.IdType,
                        CoutOfTickets =30,
                        NamePerformance="PerformanceName",
                        Price =100,
                        TypeName ="type"
                    };
                });

            _ticketOptionsService = new
TicketOptionsService(_ticketOptionsRepositoryMock.Object, _mapperMock.Object);

            var result = await
_ticketOptionsService.GetAllAvailableTickets(request);

            Assert.NotNull(result);
            Assert.NotEmpty(result);
        }

        [Fact]
        public async Task
GetAllAvailableTickets_WhenTicketsFound_ShouldReturnsExeption()
        {
            var request = new GetOptionsRequest { IdPerformance = 1 };

            List<AllTicket> allTickets = new List<AllTicket>();
            _ticketOptionsRepositoryMock.Setup(repo =>
repo.GetAllTickets(request.IdPerformance))
                .ReturnsAsync(allTickets);

            _ticketOptionsService = new
TicketOptionsService(_ticketOptionsRepositoryMock.Object, _mapperMock.Object);

            var exception = await Assert.ThrowsAsync<Exception>(async () => await
_ticketOptionsService.GetAllAvailableTickets(request));

            Assert.Equal($"Квитків з id={request.IdPerformance} не знайдено",
exception.Message);
        }

        [Fact]
        public async Task
GetIdPerformanceInOption_WhenOptionNotFound_ThrowsException()
        {
            int idOption = 1;

            _ticketOptionsRepositoryMock.Setup(repo => repo.GetByIdAsync(idOption))
                .ReturnsAsync(() => null);
            _ticketOptionsService = new
TicketOptionsService(_ticketOptionsRepositoryMock.Object, _mapperMock.Object);

            var exception = await Assert.ThrowsAsync<Exception>(async () => await
_ticketOptionsService.GetIdPerformanceInOption(idOption));
```

```
            Assert.Equal($"No Option with Id {idOption}", exception.Message);
        }

        [Fact]
        public async Task
GetIdPerformanceInOption_WhenOptionFound_ReturnsIdPerformance()
        {
            int idOption = 1;
            int expectedIdPerformance = 10;

            var option = new AllTicket { Id = idOption, IdPerformance =
expectedIdPerformance };

            _ticketOptionsRepositoryMock.Setup(repo => repo.GetByIdAsync(idOption))
                .ReturnsAsync(option);

            _ticketOptionsService = new
TicketOptionsService(_ticketOptionsRepositoryMock.Object, _mapperMock.Object);

            var expectedidPerformance = await
_ticketOptionsService.GetIdPerformanceInOption(idOption);

            Assert.Equal(expectedIdPerformance, expectedidPerformance);
        }
    }
}
```

```
using AutoMapper;
using Azure.Core;
using BoxTicketApi.BLL.Requests.Ticket;
using BoxTicketApi.BLL.Responses.Author;
using BoxTicketApi.BLL.Responses.Performance;
using BoxTicketApi.BLL.Responses.Ticket;
using BoxTicketApi.BLL.Services;
using BoxTicketApi.BLL.Services.Base;
using BoxTicketApi.DAL.Entities;
using BoxTicketApi.DAL.Repositories.Base;
using Moq;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;


namespace BoxTicketApi.Test.BLL.Tests
{
```

```
public class TicketServiceTests
{
    private readonly Mock<IMapper> _mapperMock = new();
    private readonly Mock<ITicketRepository> _ticketRepositoryMock = new();
    private TicketService _ticketService = null!;

    [Fact]
    public async Task BookTicket_WhenSeatAvailable_ReturnsTicketIdResponse()
    {
        TicketIdResponse response = new TicketIdResponse() { idTicket=1};
        Ticket ticket = new Ticket();
        var request = new TicketReqest{IdTicketOptions = 1,SeatNumber = 5,IdUser = 123};
        int idPerformance = 1;
        var boughtSeats = new List<int> { 1, 3, 7 };

        _mapperMock.Setup(m => m.Map<TicketIdResponse>(It.IsAny<Ticket>()))
          .Returns(response);
        _ticketRepositoryMock.Setup(repo => repo.GetBoughtSeatsByType(idPerformance, request.IdTicketOptions))
           .ReturnsAsync(boughtSeats);
        _ticketRepositoryMock.Setup(repo => repo.AddAsync(It.IsAny<Ticket>()))
           .Callback<Ticket>(ticket =>{ticket.Id = 1;})
           .Returns(Task.CompletedTask);
        _ticketService = new TicketService(_ticketRepositoryMock.Object, _mapperMock.Object);

        var result = await _ticketService.BookTicket(request, idPerformance);

        Assert.NotNull(result);
        Assert.Equal(1, result.idTicket);
```

```
    }

    [Fact]
    public async Task BookTicket_WhenSeatAlreadyBooked_ThrowsException()
    {
        TicketIdResponse response = new TicketIdResponse() { idTicket = 1 };
        var request = new TicketReqest { IdTicketOptions = 1, SeatNumber = 3, IdUser = 123 };
        int idPerformance = 1;
        var boughtSeats = new List<int> { 1, 3, 7 };

        _ticketRepositoryMock.Setup(repo => repo.GetBoughtSeatsByType(idPerformance, request.IdTicketOptions))
            .ReturnsAsync(boughtSeats);
        _mapperMock.Setup(m => m.Map<TicketIdResponse>(It.IsAny<Ticket>()))
            .Returns(response);
        _ticketService = new TicketService(_ticketRepositoryMock.Object, _mapperMock.Object);

        var exception = await Assert.ThrowsAsync<Exception>(async () => await _ticketService.BookTicket(request, idPerformance));

        Assert.Equal($"Квиток з місцем {request.SeatNumber} не доступний.", exception.Message);
    }

    [Fact]
    public async Task BuyTicket_WhenSeatAvailable_ReturnsTicketIdResponse()
    {
        TicketIdResponse response = new TicketIdResponse() { idTicket = 1 };
        Ticket ticket = new Ticket();
```

```
        var request = new TicketReqest { IdTicketOptions = 1, SeatNumber = 5, IdUser = 123 };
        int idPerformance = 1;
        var boughtSeats = new List<int> { 1, 3, 7 };

        _mapperMock.Setup(m => m.Map<TicketIdResponse>(It.IsAny<Ticket>()))
           .Returns(response);
        _ticketRepositoryMock.Setup(repo => repo.GetBoughtSeatsByType(idPerformance, request.IdTicketOptions))
           .ReturnsAsync(boughtSeats);
        _ticketRepositoryMock.Setup(repo => repo.AddAsync(It.IsAny<Ticket>()))
           .Callback<Ticket>(ticket => { ticket.Id = 1; })
           .Returns(Task.CompletedTask);
        _ticketService = new TicketService(_ticketRepositoryMock.Object, _mapperMock.Object);

        var result = await _ticketService.BuyTicket(request, idPerformance);

        Assert.NotNull(result);
        Assert.Equal(1, result.idTicket);
    }

    [Fact]
    public async Task BuyTicket_WhenSeatAlreadyBooked_ThrowsException()
    {
        TicketIdResponse response = new TicketIdResponse() { idTicket = 1 };
        var request = new TicketReqest { IdTicketOptions = 1, SeatNumber = 3, IdUser = 123 };
        int idPerformance = 1;
        var boughtSeats = new List<int> { 1, 3, 7 };
```

```
        _ticketRepositoryMock.Setup(repo => repo.GetBoughtSeatsByType(idPerformance, request.IdTicketOptions))
            .ReturnsAsync(boughtSeats);
        _mapperMock.Setup(m => m.Map<TicketIdResponse>(It.IsAny<Ticket>()))
            .Returns(response);
        _ticketService = new TicketService(_ticketRepositoryMock.Object, _mapperMock.Object);

        var exception = await Assert.ThrowsAsync<Exception>(async () => await _ticketService.BuyTicket(request, idPerformance));

        Assert.Equal($"Квиток з місцем {request.SeatNumber} не доступний.", exception.Message);
    }

    [Fact]
    public async Task BuyBookedTicket_WhenTicketFoundAndBelongsToUser_ReturnsTicketIdResponse()
    {
        var request = new TicketByIdReqest{ Id = 1,IdUser = 1};
        var ticket = new Ticket{Id = 1,IdUser = request.IdUser};
        var ticketExpected = new TicketIdResponse { idTicket=1};

        _ticketRepositoryMock.Setup(repo => repo.GetByIdAsync(request.Id))
            .ReturnsAsync(ticket);
        _ticketRepositoryMock.Setup(repo => repo.UpdateAsync(It.IsAny<Ticket>()))
            .Callback<Ticket>(ticketToUpdate =>
            {
                ticketToUpdate.IdStatus = 1;  });
```

```
        _ticketService = new TicketService(_ticketRepositoryMock.Object,
_mapperMock.Object);
        _mapperMock.Setup(m => m.Map<TicketIdResponse>(It.IsAny<Ticket>()))
            .Returns(ticketExpected);

        var result = await _ticketService.BuyBookedTicket(request);

        Assert.NotNull(result);
        Assert.Equal(request.Id, result.idTicket);
    }

    [Fact]
    public async Task BuyBookedTicket_WhenTicketNotFound_ThrowsException()
    {
        var request = new TicketByIdReqest
        {
            Id = 1,
            IdUser = 2
        };

        _ticketRepositoryMock.Setup(repo => repo.GetByIdAsync(request.Id))
            .ReturnsAsync(() => null);
        _ticketService = new TicketService(_ticketRepositoryMock.Object,
_mapperMock.Object);

        var exception = await Assert.ThrowsAsync<Exception>(async () => await
_ticketService.BuyBookedTicket(request));

        Assert.Equal($"Квиток з id {request.Id} не знайдений.", exception.Message);
    }
```

```
    [Fact]
    public async Task BuyBookedTicket_WhenTicketBelongsToAnotherUser_ThrowsException()
    {
        var request = new TicketByIdReqest
        {
            Id = 1,
            IdUser = 1
        };

        var ticket = new Ticket
        {
            Id = 1,
            IdUser = 2
        };

        _ticketRepositoryMock.Setup(repo => repo.GetByIdAsync(request.Id))
            .ReturnsAsync(ticket);
        _ticketService = new TicketService(_ticketRepositoryMock.Object, _mapperMock.Object);
        var exception = await Assert.ThrowsAsync<Exception>(async () => await _ticketService.BuyBookedTicket(request));

        Assert.Equal($"Квиток з id {request.Id} бронював інший користувач.", exception.Message);
    }


    [Fact]
```

```
    public async Task CancelBookedTicket_WhenTicketFoundAndBelongsToUser_ReturnsTicketIdResponse()
    {
        var request = new TicketByIdReqest { Id = 1, IdUser = 1 };
        var ticket = new Ticket { Id = 1, IdUser = request.IdUser };
        var ticketExpected = new TicketIdResponse { idTicket = 1 };

        _mapperMock.Setup(m => m.Map<TicketIdResponse>(It.IsAny<TicketByIdReqest>()))
            .Returns(ticketExpected);

        _ticketRepositoryMock.Setup(repo => repo.GetByIdAsync(request.Id))
            .ReturnsAsync(ticket);

        _ticketService = new TicketService(_ticketRepositoryMock.Object, _mapperMock.Object);

        var result = await _ticketService.CancelBookedTicket(request);

        Assert.NotNull(result);
        Assert.Equal(request.Id, result.idTicket);
    }

    [Fact]
    public async Task CancelBookedTicket_WhenTicketNotFound_ThrowsException()
    {
        var request = new TicketByIdReqest
        {
            Id = 1,
```

```
            IdUser = 123
        };

        _ticketRepositoryMock.Setup(repo => repo.GetByIdAsync(request.Id))
            .ReturnsAsync(() => null);

        _ticketService = new TicketService(_ticketRepositoryMock.Object, _mapperMock.Object);

        var exception = await Assert.ThrowsAsync<Exception>(async () => await _ticketService.CancelBookedTicket(request));

        Assert.Equal($"Квиток з id {request.Id} не знайдений.", exception.Message);
    }

    [Fact]
    public async Task CancelBookedTicket_WhenTicketBelongsToAnotherUser_ThrowsException()
    {
        var request = new TicketByIdReqest
        {
            Id = 1,
            IdUser = 1
        };

        var ticket = new Ticket
        {
            Id = 1,
            IdUser = 2,
        };
```

```
        _ticketRepositoryMock.Setup(repo => repo.GetByIdAsync(request.Id))
            .ReturnsAsync(ticket);

        _ticketService = new TicketService(_ticketRepositoryMock.Object, _mapperMock.Object);

        var exception = await Assert.ThrowsAsync<Exception>(async () => await _ticketService.CancelBookedTicket(request));

        Assert.Equal($"Квиток з id {request.Id} бронював інший користувач.", exception.Message);
    }

    [Fact]
    public async Task GetAllTickets_ReturnsListOfTickets()
    {
        int userId = 1;
        var ticketsList = new List<Ticket>
        {
            new Ticket { Id = 1, IdUser = userId},
            new Ticket { Id = 2, IdUser = userId},
        };

        var ticketsListExpected = new List<TicketResponse>
        {
            new TicketResponse{ Id =1, IdUser=userId, IdPerformance=1, Price=100, SeatNumber=4, Status="Проданий", IdTicketOptions=1, Performance="p"},
            new TicketResponse{ Id =1, IdUser=userId, IdPerformance=2, Price=200, SeatNumber=1, Status="Проданий", IdTicketOptions=3, Performance="p"}
        };
```

```
        _ticketRepositoryMock.Setup(repo => repo.GetAllTicketsById(userId))
            .ReturnsAsync(ticketsList);
        _mapperMock.Setup(m => m.Map<List<TicketResponse>>(It.IsAny<List<Ticket>>()))
            .Returns(ticketsListExpected);
        _ticketService = new TicketService(_ticketRepositoryMock.Object, _mapperMock.Object);

        var result = await _ticketService.GetAllTickets(userId);

        Assert.NotNull(result);
        Assert.Equal(ticketsList.Count, result.Count);
    }

  }
}
using AutoMapper;
using Azure.Core;
using BoxTicketApi.BLL.Mapper;
using BoxTicketApi.BLL.Requests.Auth;
using BoxTicketApi.BLL.Requests.Genre;
using BoxTicketApi.BLL.Responses.Auth;
using BoxTicketApi.BLL.Responses.Genre;
using BoxTicketApi.BLL.Services;
using BoxTicketApi.BLL.Services.Base;
using BoxTicketApi.DAL.Contexts;
using BoxTicketApi.DAL.Entities;
using BoxTicketApi.DAL.Repositories.Base;
using Microsoft.Extensions.Configuration;
using Microsoft.IdentityModel.Tokens;
using Moq;
```

```
using Newtonsoft.Json.Linq;
using System;
using System.Collections.Generic;
using System.IdentityModel.Tokens.Jwt;
using System.Linq;
using System.Security.Claims;
using System.Security.Cryptography;
using System.Security.Policy;
using System.Text;
using System.Threading.Tasks;

namespace BoxTicketApi.Test.BLL.Tests
{
    public class UserServiceTests
    {
        private readonly Mock<IMapper> _mapperMock = new();
        private readonly Mock<IUserRepository> _userRepositoryMock = new();
        private readonly Mock<IRefreshTokenRepository> _refreshTokenRepositoryMock = new();
        private readonly Mock<IConfiguration> _configMock = new();
        private UserService _userService = null!;

        [Fact]
        public async Task RegisterUserAsync_WhenUserDoesNotExist_ReturnsAuthResponse()
        {
            var signUpRequest = new SignUpRequest
            {
                FirstName = "first",
                MiddleName = "middle",
                LastName = "Last",
```

```
            Email = "email",
            Password = "Password"
        };
        Random rnd = new Random();
        byte[] hash = new byte[32];
        rnd.NextBytes(hash);

        var user = new UserAccount
        {
            Id = 1,
            FirstName = "first",
            MiddleName = "middle",
            LastName = "Last",
            Email = "email",
            PasswordHash = hash,
            PasswordSalt = hash,
            IdRole = 1
        };

        var authResponse = new AuthResponse
        {
            Email="email",
            Password="password"
        };
        var token = new RefreshToken
        {
            IdUser = 1
        };
        var jwtHandlerMock = new Mock<JwtSecurityTokenHandler>();
        _userRepositoryMock.Setup(repo                                    =>
repo.GetUserByEmailAsync(signUpRequest.Email))
```

```
            .ReturnsAsync((UserAccount)null);
        _userRepositoryMock.Setup(repo => repo.AddAsync(It.IsAny<UserAccount>()))
            .Callback<UserAccount>(user => { user.Id = 1; })
            .Returns(Task.CompletedTask);
        _refreshTokenRepositoryMock.Setup(repo => repo.AddAsync(It.IsAny<RefreshToken>()))
            .Callback<RefreshToken>(token => { token.Id = 1; })
            .Returns(Task.CompletedTask);
        _mapperMock.Setup(m => m.Map<AuthResponse>(signUpRequest))
            .Returns(authResponse);
        _userService = new UserService(_userRepositoryMock.Object, _configMock.Object, _mapperMock.Object, _refreshTokenRepositoryMock.Object, jwtHandlerMock.Object);

        var result = await _userService.RegisterUserAsync(signUpRequest);

        Assert.NotNull(result);
    }

    [Fact]
    public async Task RegisterUserAsync_WhenUserExists_ThrowsException()
    {
        var signUpRequest = new SignUpRequest
        {
            FirstName = "first",
            MiddleName = "middle",
            LastName = "Last",
            Email = "email",
            Password = "Password"
        };
```

```
        Random rnd = new Random();
        byte[] hash = new byte[32];
        rnd.NextBytes(hash);
        var existingUser = new UserAccount
        {
            Id = 1,
            FirstName = "first",
            MiddleName = "middle",
            LastName = "Last",
            Email = "email",
            PasswordHash = hash,
            PasswordSalt = hash,
            IdRole = 1
        };
        var jwtHandlerMock = new Mock<JwtSecurityTokenHandler>();
        _userRepositoryMock.Setup(repo => repo.GetUserByEmailAsync(signUpRequest.Email))
            .ReturnsAsync(existingUser);
        _userService = new UserService(_userRepositoryMock.Object, _configMock.Object, _mapperMock.Object, _refreshTokenRepositoryMock.Object, jwtHandlerMock.Object);

        var exception = await Assert.ThrowsAsync<Exception>(async () => await _userService.RegisterUserAsync(signUpRequest));
        Assert.Equal("User already exists.", exception.Message);
    }

    [Fact]
    public async Task RefreshToken_WhenValidRefreshToken_ReturnsTokenResponse()
    {
```

```
        int userId = 1;
        string refreshToken = "valid_refresh_token";

        var tokenUser = new RefreshToken
        {
            Token = refreshToken,
            Expires = DateTime.Now.AddDays(1)
        };

        var role = new RoleUser
        {
            Id = 1,
            NameRole = "User"
        };
        var existingUser = new UserAccount
        {
            IdRole = userId,
            Id = userId,
            FirstName = "first",
            MiddleName = "middle",
            LastName = "Last",
            Email = "email",
            IdRoleNavigation = role
        };
        var jwtHandlerMock = new Mock<JwtSecurityTokenHandler>();
        jwtHandlerMock.Setup(x => x.WriteToken(It.IsAny<SecurityToken>()))
            .Returns("MockedJWTToken");
        _configMock.Setup(x                                                    =>
x.GetSection("AppSettings:Token").Value).Returns("your_token_value");
        _refreshTokenRepositoryMock.Setup(repo                                 =>
repo.GetRefreshTokenByUser(userId))
```

```
            .ReturnsAsync(tokenUser);
        _userRepositoryMock.Setup(repo => repo.GetByIdAsync(userId))
            .ReturnsAsync(existingUser);
        _userService = new UserService(_userRepositoryMock.Object, _configMock.Object, _mapperMock.Object, _refreshTokenRepositoryMock.Object, jwtHandlerMock.Object);

        var result = await _userService.RefreshToken(refreshToken, userId);

        Assert.NotNull(result);
    }

    [Fact]
    public async Task RefreshToken_WhenInvalidRefreshToken_ReturnsExeption()
    {
        int userId = 1;
        string refreshTokenInvalid = "invalid_refresh_token";
        string refreshToken = "valid_refresh_token";

        var tokenUserValid = new RefreshToken
        {
            Token = refreshToken,
            Expires = DateTime.Now.AddDays(1)
        };
        var tokenUserInValid = new RefreshToken
        {
            Token = refreshToken,
            Expires = DateTime.Now.AddDays(1)
        };

        var existingUser = new UserAccount
```

```
        {
            Id = userId
        };
        var jwtHandlerMock = new Mock<JwtSecurityTokenHandler>();
        _refreshTokenRepositoryMock.Setup(repo => repo.GetRefreshTokenByUser(userId))
            .ReturnsAsync(tokenUserValid);
        _userRepositoryMock.Setup(repo => repo.GetByIdAsync(userId))
            .ReturnsAsync(existingUser);
        _userService = new UserService(_userRepositoryMock.Object, _configMock.Object, _mapperMock.Object, _refreshTokenRepositoryMock.Object, jwtHandlerMock.Object);

        var exception = await Assert.ThrowsAsync<Exception>(async () => await _userService.RefreshToken(refreshTokenInvalid, userId));

        Assert.Equal($"Invalid Refresh Token.", exception.Message);
    }

    [Fact]
    public async Task RefreshToken_WhenInvalidExpiresDate_ReturnsExeption()
    {
        int userId = 1;
        string refreshToken = "valid_refresh_token";

        var tokenUserValid = new RefreshToken
        {
            Token = refreshToken,
            Expires = new DateTime (2023,1,1)
        };
```

```
        var existingUser = new UserAccount
        {
            Id = userId
        };
        var jwtHandlerMock = new Mock<JwtSecurityTokenHandler>();
        _refreshTokenRepositoryMock.Setup(repo => repo.GetRefreshTokenByUser(userId))
            .ReturnsAsync(tokenUserValid);
        _userRepositoryMock.Setup(repo => repo.GetByIdAsync(userId))
            .ReturnsAsync(existingUser);
        _userService = new UserService(_userRepositoryMock.Object, _configMock.Object, _mapperMock.Object, _refreshTokenRepositoryMock.Object, jwtHandlerMock.Object);

        var exception = await Assert.ThrowsAsync<Exception>(async () => await _userService.RefreshToken(refreshToken, userId));

        Assert.Equal($"Token expired.", exception.Message);
    }

    [Fact]
    public async Task Login_WhenValidPassword_ReturnsTokenResponse()
    {
        int idUser = 1;
        string tokenUser = "tokenUser";
        string tokenUpdate = "tokenUser";
        string password = "passwordUser";
        byte[] passwordHash;
        byte[] passwordSalt;

        using (var hmac = new HMACSHA512())
```

```
        {
            passwordSalt = hmac.Key;
            passwordHash                                                    =
hmac.ComputeHash(System.Text.Encoding.UTF8.GetBytes(password));
        }
        var signInRequest = new SignInRequest
        {
            Email = "string",
            Password = password
        };
        var role = new RoleUser
        {
            Id = 1,
            NameRole = "User"
        };
        var existingUser = new UserAccount
        {
            Id = idUser,
            FirstName = "first",
            MiddleName = "middle",
            LastName = "Last",
            Email = "email",
            PasswordHash = passwordHash,
            PasswordSalt = passwordSalt,
            IdRoleNavigation= role,
            IdRole = 1
        };
        var token = new RefreshToken
        {
            Token = tokenUser,
            Expires = DateTime.Now.AddDays(1)
```

```
        };
        var jwtHandlerMock = new Mock<JwtSecurityTokenHandler>();
        _configMock.Setup(x => x.GetSection("AppSettings:Token").Value).Returns("your_token_value");
        _userRepositoryMock.Setup(repo => repo.GetUserByEmailAsync(signInRequest.Email))
            .ReturnsAsync(existingUser);
        _refreshTokenRepositoryMock.Setup(repo => repo.GetRefreshTokenByUser(idUser))
            .ReturnsAsync(token);
        _userService = new UserService(_userRepositoryMock.Object, _configMock.Object, _mapperMock.Object, _refreshTokenRepositoryMock.Object, jwtHandlerMock.Object);

        var result = await _userService.Login(signInRequest);

        Assert.NotNull(result);
        Assert.Equal(idUser, result.UserId);
    }

    [Fact]
    public async Task Login_WhenInValidEmail_ReturnExeption()
    {
        int idUser = 1;
        string tokenUser = "tokenUser";
        string tokenUpdate = "tokenUser";
        string password = "passwordUser";
        byte[] passwordHash;
        byte[] passwordSalt;

        using (var hmac = new HMACSHA512())
```

```
        {
            passwordSalt = hmac.Key;
            passwordHash                                                    =
hmac.ComputeHash(System.Text.Encoding.UTF8.GetBytes(password));
        }
        var signInRequest = new SignInRequest
        {
            Email = "string",
            Password = password
        };
        var role = new RoleUser
        {
            Id = 1,
            NameRole = "User"
        };
        var existingUser = new UserAccount
        {
            Id = idUser,
            FirstName = "first",
            MiddleName = "middle",
            LastName = "Last",
            Email = "email",
            PasswordHash = passwordHash,
            PasswordSalt = passwordSalt,
            IdRoleNavigation = role,
            IdRole = 1
        };
        var token = new RefreshToken
        {
            Token = tokenUser,
            Expires = DateTime.Now.AddDays(1)
```

```
        };
        var jwtHandlerMock = new Mock<JwtSecurityTokenHandler>();
        UserAccount userNull = null;
        _configMock.Setup(x => x.GetSection("AppSettings:Token").Value).Returns("your_token_value");
        _userRepositoryMock.Setup(repo => repo.GetUserByEmailAsync(signInRequest.Email))
            .ReturnsAsync(userNull);
        _refreshTokenRepositoryMock.Setup(repo => repo.GetRefreshTokenByUser(idUser))
            .ReturnsAsync(token);
        _userService = new UserService(_userRepositoryMock.Object, _configMock.Object, _mapperMock.Object, _refreshTokenRepositoryMock.Object, jwtHandlerMock.Object);

        var exception = await Assert.ThrowsAsync<Exception>(async () => await _userService.Login(signInRequest));

        Assert.Equal($"User doesn't exist with this email: {signInRequest.Email}.", exception.Message);
    }

    [Fact]
    public async Task Login_WhenInValidPassword_ReturnExeption()
    {
        int idUser = 1;
        string tokenUser = "tokenUser";
        string tokenUpdate = "tokenUser";
        string password = "passwordUser";
        string passwordInvalid = "invalidPasswordUser";
```

```
        byte[] passwordHash;
        byte[] passwordSalt;

        using (var hmac = new HMACSHA512())
        {
            passwordSalt = hmac.Key;
            passwordHash                                                    =
hmac.ComputeHash(System.Text.Encoding.UTF8.GetBytes(password));
        }
        var signInRequest = new SignInRequest
        {
            Id = idUser,
            Email = "email",
            Password = passwordInvalid
        };
        var role = new RoleUser
        {
            Id = 1,
            NameRole = "User"
        };
        var existingUser = new UserAccount
        {
            Id = idUser,
            FirstName = "first",
            MiddleName = "middle",
            LastName = "Last",
            Email = "email",
            PasswordHash = passwordHash,
            PasswordSalt = passwordSalt,
            IdRoleNavigation = role,
            IdRole = 1
```

```
        };
        var token = new RefreshToken
        {
            Token = tokenUser,
            Expires = DateTime.Now.AddDays(1)
        };
        var jwtHandlerMock = new Mock<JwtSecurityTokenHandler>();
        _configMock.Setup(x => x.GetSection("AppSettings:Token").Value).Returns("your_token_value");
        _userRepositoryMock.Setup(repo => repo.GetUserByEmailAsync(signInRequest.Email))
            .ReturnsAsync(existingUser);
        _refreshTokenRepositoryMock.Setup(repo => repo.GetRefreshTokenByUser(idUser))
            .ReturnsAsync(token);
        _userService = new UserService(_userRepositoryMock.Object, _configMock.Object, _mapperMock.Object, _refreshTokenRepositoryMock.Object, jwtHandlerMock.Object);

        var exception = await Assert.ThrowsAsync<Exception>(async () => await _userService.Login(signInRequest));

        Assert.Equal($"Wrong password.", exception.Message);
    }
  }
}
```

```
using BoxTicketApi.DAL.Repositories.Base;
using BoxTicketApi.DAL.Repositories;
using BoxTicketApi.Test.DAL.Test;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace BoxTicketApi.Test.DAL.Test
{
```

```
    public class AuthorRepositoryTests : IClassFixture<DatabaseFixture>
    {
        private readonly DatabaseFixture _fixture;
        private readonly IAuthorRepository _authorRepository;

        public AuthorRepositoryTests(DatabaseFixture fixture)
        {
            _fixture = fixture;
            _authorRepository = new AuthorRepository(_fixture.Context);
        }

        [Fact]
        public async Task GetAllAsync_ShouldreturnListWithAuthor()
        {
            var author = await _authorRepository.GetAllAsync();

            Assert.NotNull(author);
        }
    }
}


using BoxTicketApi.DAL.Contexts;
using BoxTicketApi.DAL.Entities;
using BoxTicketApi.DAL.Repositories;
using Microsoft.EntityFrameworkCore;
using Moq;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Dasync.Collections;
using BoxTicketApi.DAL.Repositories.Base;

namespace BoxTicketApi.Test.DAL.Test
{
    public class DatabaseFixture: IDisposable
    {
        public BoxTicketContext Context;

        public DatabaseFixture()
        {
            var options = new DbContextOptionsBuilder<BoxTicketContext>()
                .UseInMemoryDatabase(databaseName: Guid.NewGuid().ToString())
                .Options;

            Context = new BoxTicketContext(options);
            SeedTestData().Wait();
        }

        private async Task SeedTestData()
        {
            byte[] pass = new byte[10];
            byte[] salt = new byte[5];

            Random random = new Random();

            List<RoleUser> usersRole = new List<RoleUser>()
            {
                new RoleUser() { NameRole = "Admin"},
                new RoleUser() { NameRole = "User" }
            };
            await Context.RoleUsers.AddRangeAsync(usersRole);

            List<StatusTicket> statusTickets = new List<StatusTicket>
            {
                new StatusTicket() { StatusName = "Куплений" },
                new StatusTicket() { StatusName = "Заброньований"}
            };
```

```
            await Context.StatusTickets.AddRangeAsync(statusTickets);

            List<TypeOfTicket> types = new List<TypeOfTicket>()
            {
                new TypeOfTicket() { TypeName = "Звичайний" },
                new TypeOfTicket() { TypeName = "Покращений" },
                new TypeOfTicket() { TypeName = "Люкс" }
            };
            await Context.TypeOfTickets.AddRangeAsync(types);

            List<Author> authors = new List<Author>()
            {
                new Author { FirstName = "Анастасія", MiddleName = "Михайлівна",
LastName = "Сидоренко" },
                new Author { FirstName = "Ігор", MiddleName = "Іванович", LastName =
"Уманчук" }
            };
            await Context.Authors.AddRangeAsync(authors);

            List<Genre> genres = new List<Genre>()
            {
                new Genre { NameGenre = "Драма" },
                new Genre { NameGenre = "Фікшн" }
            };
            await Context.Genres.AddRangeAsync(genres);

            random.NextBytes(pass);
            random.NextBytes(salt);
            UserAccount userAccount1 = new UserAccount() { FirstName = "first1",
MiddleName = "middle1", LastName = "last1", Email = "email1", IdRole = 2,
PasswordHash = pass, PasswordSalt= salt};
            random.NextBytes(pass);
            random.NextBytes(salt);
            UserAccount userAccount2 = new UserAccount() { FirstName = "first2",
MiddleName = "middle2", LastName = "last2", Email = "email2", IdRole = 1,
PasswordHash = pass, PasswordSalt = salt };
            await Context.UserAccounts.AddAsync(userAccount1);
            await Context.UserAccounts.AddAsync(userAccount2);
            await Context.SaveChangesAsync();

            List<RefreshToken> refreshTokens = new List<RefreshToken>()
            {
                new RefreshToken() { Token="token1", Expires = DateTime.Now,
IdUser=1},
                new RefreshToken() { Token = "token2", Expires = DateTime.Now,
IdUser = 2 }
            };
            await Context.RefreshTokens.AddRangeAsync(refreshTokens);

            List<Performance> performances = new List<Performance>()
            {
                new Performance{PerformanceName = "Назва1", DateTimeEvent = new
DateTime (2023, 1, 2), IdAuthor=1, IdGenre=2},
                new Performance { PerformanceName = "Назва2", DateTimeEvent = new
DateTime(2023, 5, 8), IdAuthor = 2, IdGenre = 1}
            };
            await Context.Performances.AddRangeAsync(performances);
            await Context.SaveChangesAsync();

            List<AllTicket> options = new List<AllTicket>()
            {
                new AllTicket() { IdType = 1, IdPerformance = 1, CoutOfTickets = 50,
Price = 100},
                new AllTicket() { IdType = 2, IdPerformance = 1, CoutOfTickets = 30,
Price = 200 },
                new AllTicket() { IdType = 3, IdPerformance = 1, CoutOfTickets = 20,
Price = 300 },
                new AllTicket() { IdType = 1, IdPerformance = 2, CoutOfTickets = 60,
Price = 100 },
```

```
                new AllTicket() { IdType = 2, IdPerformance = 2, CoutOfTickets = 20,
Price = 200 }
            };
            await Context.AllTickets.AddRangeAsync(options);
            await Context.SaveChangesAsync();

            List<Ticket> tickets = new List<Ticket>()
            {
                new Ticket() { IdAllTickets = 1, IdUser = 1, IdStatus = 1,
SeatNumber = 2 },
                new Ticket() { IdAllTickets = 2, IdUser = 1, IdStatus = 1,
SeatNumber = 4 },
                new Ticket() { IdAllTickets = 3, IdUser = 1, IdStatus = 2,
SeatNumber = 6 },
                new Ticket() { IdAllTickets = 4, IdUser = 1, IdStatus = 2,
SeatNumber = 3 }
            };
            await Context.Tickets.AddRangeAsync(tickets);
            await Context.SaveChangesAsync();
        }

        public void Dispose()
        {
            Context.Dispose();
        }
    }
}
using BoxTicketApi.DAL.Repositories.Base;
using BoxTicketApi.DAL.Repositories;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using BoxTicketApi.Test.DAL.Test;
using Xunit;
using BoxTicketApi.DAL.Entities;

namespace BoxTicketApi.Test.DAL.Test
{
    public class GenreRepositoryTests: IClassFixture<DatabaseFixture>
    {
        private readonly DatabaseFixture _fixture;
        private readonly IGenreRepository _genreRepository;

        public GenreRepositoryTests(DatabaseFixture fixture)
        {
            _fixture = fixture;
            _genreRepository = new GenreRepository(_fixture.Context);
        }

        [Fact]
        public async Task
DeleteGenre_WithValidGenreId_ShouldDeleteGenreAndChangeCount()
        {
            var genres = await _genreRepository.GetAllAsync();
            await _genreRepository.DeleteAsync(1);
            var genresAfter = await _genreRepository.GetAllAsync();
            var ifExist = await _genreRepository.GetByIdAsync(1);

            Assert.Null(ifExist);
            Assert.NotEqual(genres.Count, genresAfter.Count);
        }

        [Fact]
        public async Task AddGenre_WithValidGenre_ShouldAddGenreAndChangeCount()
        {
            Genre genreobject = new Genre { NameGenre = "Драма" };

            var genres = await _genreRepository.GetAllAsync();
```

```
            await _genreRepository.AddAsync(genreobject);
            var genresAfter = await _genreRepository.GetAllAsync();
            var ifExist = await _genreRepository.GetByIdAsync(genreobject.Id);

            Assert.NotNull(ifExist);
            Assert.NotEqual(genres.Count, genresAfter.Count);
        }
    }
}
using BoxTicketApi.DAL.Contexts;
using BoxTicketApi.DAL.Entities;
using BoxTicketApi.DAL.Repositories;
using BoxTicketApi.DAL.Repositories.Base;
using BoxTicketApi.Test.DAL.Test;
using Dasync.Collections;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Query;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Linq.Expressions;
using System.Text;
using System.Threading.Tasks;
using Xunit;

namespace BoxTicketApi.Test.DAL.Test
{
    public class PerformanceRepositoryTests : IClassFixture<DatabaseFixture>
    {
        private readonly DatabaseFixture _fixture;
        private readonly IPerformanceRepository _performanceRepository;

        public PerformanceRepositoryTests(DatabaseFixture fixture)
        {
            _fixture = fixture;
            _performanceRepository = new PerformanceRepository(_fixture.Context);
        }

        [Fact]
        public async Task
GetPerformancesByAuthor_ShouldReturnPerformancesForGivenAuthor()
        {
            var performances = await
_performanceRepository.GetPerformancesByAuthor(1);

            Assert.NotNull(performances);
            Assert.Equal(1, performances.Count);
        }

        [Fact]
        public async Task
GetPerformancesByAuthor_ShouldReturnPerformancesForGivenDate()
        {
            var performances = await
_performanceRepository.GetPerformancesByDate(new DateTime(2023,1,2));

            Assert.NotNull(performances);
            Assert.Equal(1, performances.Count);
        }

        [Fact]
        public async Task
GetPerformancesByAuthor_ShouldReturnPerformancesForGivenGenre()
        {
            var performances = await
_performanceRepository.GetPerformancesByGenre(2);

            Assert.NotNull(performances);
```

```
            Assert.Equal(1, performances.Count);
        }

        [Fact]
        public async Task
GetPerformancesByAuthor_ShouldReturnPerformancesForGivenName()
        {
            var performances = await
_performanceRepository.GetPerformancesByName("Назва1");

            Assert.NotNull(performances);
            Assert.Equal(1, performances.Count);
        }
    }
}
using BoxTicketApi.DAL.Repositories.Base;
using BoxTicketApi.DAL.Repositories;
using BoxTicketApi.Test.DAL.Test;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using BoxTicketApi.DAL.Entities;

namespace BoxTicketApi.Test.DAL.Test
{
    public class RefreshTokenRepositoryTests : IClassFixture<DatabaseFixture>
    {
        private readonly DatabaseFixture _fixture;
        private readonly IRefreshTokenRepository _refreshTokenRepository;

        public RefreshTokenRepositoryTests(DatabaseFixture fixture)
        {
            _fixture = fixture;
            _refreshTokenRepository = new RefreshTokenRepository(_fixture.Context);
        }

        [Fact]
        public async Task GetRefreshTokenByUser_WithUserId_ShouldGetRefreshToken()
        {
            int userId = 1;
            var token = await _refreshTokenRepository.GetRefreshTokenByUser(userId);

            Assert.NotNull(token);
        }

        [Fact]
        public async Task UpdateAsync_ShouldUpdateRefreshToken()
        {
            string tokenString = "token11";
            RefreshToken token =  new RefreshToken() { Token = tokenString, Expires
= DateTime.Now };

            await _refreshTokenRepository.UpdateAsync(token);

            Assert.Equal(token.Token, tokenString);
        }
    }
}


using BoxTicketApi.DAL.Repositories.Base;
using BoxTicketApi.DAL.Repositories;
using BoxTicketApi.Test.DAL.Test;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
```

```
namespace BoxTicketApi.Test.DAL.Test
{
    public class TicketOptionsRepositoryTests : IClassFixture<DatabaseFixture>
    {
        private readonly DatabaseFixture _fixture;
        private readonly ITicketOptionsRepository _ticketOptionsRepository;

        public TicketOptionsRepositoryTests(DatabaseFixture fixture)
        {
            _fixture = fixture;
            _ticketOptionsRepository = new
TicketOptionsRepository(_fixture.Context);
        }

        [Fact]
        public async Task
GetAllTickets_WithIdPerformance_ShouldGetListWithTicketOptions()
        {
            int performanceId = 2;

            var performance = await
_ticketOptionsRepository.GetAllTickets(performanceId);

            Assert.NotNull(performance);
            Assert.Equal(2, performance.Count);
        }

        [Fact]
        public async Task
GetBoughtSeatsByType_WithIdPerformanceAndIdOption_ShouldReturnCountOfSeats()
        {
            int performanceId = 1;
            int idOption = 2;

            var performances = await
_ticketOptionsRepository.GetBoughtSeatsByType(performanceId, idOption);

            Assert.NotNull(performances);
            Assert.Single(performances);
        }
    }
}
using BoxTicketApi.DAL.Repositories.Base;
using BoxTicketApi.DAL.Repositories;
using BoxTicketApi.Test.DAL.Test;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace BoxTicketApi.Test.DAL.Test
{
    public class TicketRepositoryTests : IClassFixture<DatabaseFixture>
    {
        private readonly DatabaseFixture _fixture;
        private readonly ITicketRepository _ticketRepository;

        public TicketRepositoryTests(DatabaseFixture fixture)
        {
            _fixture = fixture;
            _ticketRepository = new TicketsRepository(_fixture.Context);
        }

        [Fact]
        public async Task
GetBoughtSeatsByType_WithIdPerformanceAndIdOption_ShouldReturnCountOfSeats()
        {
            int performanceId = 1;
```

```
            int idOption = 2;

            var performances = await
_ticketRepository.GetBoughtSeatsByType(performanceId, idOption);

            Assert.NotNull(performances);
            Assert.Single(performances);
        }


        [Fact]
        public async Task
GetAllTicketsById_WithIdUser_ShouldReturnListWithAllTypesOfTickets()
        {
            int idUser = 1;

            var performances = await _ticketRepository.GetAllTicketsById(idUser);

            Assert.NotNull(performances);
            Assert.Equal(4, performances.Count);
        }
    }
}
using BoxTicketApi.DAL.Repositories.Base;
using BoxTicketApi.DAL.Repositories;
using BoxTicketApi.Test.DAL.Test;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace BoxTicketApi.Test.DAL.Test
{
    public class UserRepositoryTests : IClassFixture<DatabaseFixture>
    {
        private readonly DatabaseFixture _fixture;
        private readonly IUserRepository _userRepository;

        public UserRepositoryTests(DatabaseFixture fixture)
        {
            _fixture = fixture;
            _userRepository = new UserRepository(_fixture.Context);
        }

        [Fact]
        public async Task GetUserByEmailAsync_WithValidEmail_ShouldGetUser()
        {
            string email = "email2";

            var user = await _userRepository.GetUserByEmailAsync(email);

            Assert.NotNull(user);
        }

        [Fact]
        public async Task GetUserByEmailAsync_WithInValidEmail_ShouldGetNull()
        {
            string email = "email10";

            var user = await _userRepository.GetUserByEmailAsync(email);

            Assert.Null(user);
        }
    }
}
using Azure;
using Azure.Core;
using BoxTicketApi.BLL.Requests.Auth;
using BoxTicketApi.BLL.Responses.Auth;
```

```
using BoxTicketApi.BLL.Services.Base;
using BoxTicketApi.Controllers;
using ErrorOr;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Moq;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Net;
using System.Text;
using System.Threading.Tasks;

namespace BoxTicketApi.Test.Project.Test
{
    public class AuthControllerTests
    {
        private readonly Mock<IUserService> _userServiceMock = new();
        private AuthController _authController = null!;

        [Fact]
        public async Task Register_ReturnAuthResponse()
        {
            var request = new SignUpRequest()
            {
                Password = "password",
                Email = "email",
            };
            var response = new AuthResponse()
            {
                Email = "email",
                Password = "password"
            };

            _userServiceMock.Setup(ds => ds.RegisterUserAsync(request))
                .ReturnsAsync(response);
            _authController = new AuthController(_userServiceMock.Object);

            var returnedResult = await _authController.Register(request);

            var Result = Assert.IsType<OkObjectResult>(returnedResult);
            Assert.NotNull(Result.Value);
        }

        [Fact]
        public async Task Login_ReturnAuthResponse()
        {
            var request = new SignInRequest()
            {
                Password = "password",
                Email = "email",
            };
            var response = new TokenResponse()
            {
                AccessToken = "token",
                date = DateTime.Now.AddDays(1),
                UserId=1
            };

            _userServiceMock.Setup(ds => ds.Login(request))
                .ReturnsAsync(response);
            _authController = new AuthController(_userServiceMock.Object);

            var httpContext = new DefaultHttpContext();
            _authController.ControllerContext = new ControllerContext
            {
                HttpContext = httpContext
            };
            var returnedResult = await _authController.Login(request);
            var a = returnedResult.Result;
```

```
            Assert.NotNull(returnedResult.Result);
        }

        [Fact]
        public async Task RefreshToken_WithValidCookies_Returns_OkResult()
        {
            var response = new TokenResponse()
            {
                AccessToken = "token",
                date = DateTime.Now.AddDays(1),
                UserId = 1
            };

            // Arrange
            var userServiceMock = new Mock<IUserService>();
            var controller = new AuthController(userServiceMock.Object);
            var refreshTokenValue = "yourRefreshToken";
            var userIdValue = "1";
            var expectedResult = "NewToken";

            var httpContext = new DefaultHttpContext();
            httpContext.Request.Headers["Cookie"] =
$"refreshToken={refreshTokenValue}; UserId={userIdValue}";
            controller.ControllerContext = new ControllerContext
            {
                HttpContext = httpContext
            };
            _userServiceMock.Setup(ds => ds.RefreshToken(refreshTokenValue, 1))
                .ReturnsAsync(response);

            var result = await controller.RefreshToken();

            Assert.NotNull(result);
        }

        [Fact]
        public async Task RefreshToken_Returns_BadRequest_When_UserId_Not_Exists()
        {
            // Arrange
            var mockUserService = new Mock<IUserService>();

            var controller = new AuthController(mockUserService.Object);
            controller.ControllerContext = new ControllerContext();
            controller.ControllerContext.HttpContext = new DefaultHttpContext();

            // Act
            var result = await controller.RefreshToken();

            // Assert
            Assert.IsType<BadRequestResult>(result.Result);
        }
    }
}
using BoxTicketApi.BLL.Requests.Auth;
using BoxTicketApi.BLL.Requests.Author;
using BoxTicketApi.BLL.Responses.Auth;
using BoxTicketApi.BLL.Responses.Author;
using BoxTicketApi.BLL.Services.Base;
using BoxTicketApi.Controllers;
using Microsoft.AspNetCore.Mvc;
using Moq;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace BoxTicketApi.Test.Project.Test
{
    public class AuthorContollerTests
```

```
    {
        private readonly Mock<IAuthorService> _authorServiceMock = new();
        private AuthorController _authorController = null!;

        [Fact]
        public async Task AddAuthor_ReturnAuthorResponse()
        {
            var request = new AuthorRequest()
            {
                FirstName = "first",
                LastName = "last",
                MiddleName = "middle"
            };
            AuthorResponse response = new()
            {
                FirstName = "first",
                LastName = "last",
                MiddleName = "middle",
                Id = 1
            };

            _authorServiceMock.Setup(ds => ds.AddAuthor(request))
                .ReturnsAsync(response);
            _authorController = new AuthorController(_authorServiceMock.Object);

            var returnedResult = await _authorController.AddAuthor(request);

            var Result = Assert.IsType<OkObjectResult>(returnedResult);
            Assert.NotNull(Result.Value);
        }

        [Fact]
        public async Task GetAllAuthor_ReturnAuthorResponse()
        {
            List<AuthorResponse> response = new()
            {
                new()
                {
                    FirstName="first",
                    LastName="last",
                    MiddleName="middle",
                    Id =1
                }
            };

            _authorServiceMock.Setup(ds => ds.GetAllAuthor())
                .ReturnsAsync(response);
            _authorController = new AuthorController(_authorServiceMock.Object);

            var returnedResult = await _authorController.GetAllAuthor();

            var Result = Assert.IsType<OkObjectResult>(returnedResult);
            Assert.NotNull(Result.Value);
        }
    }
}
using Microsoft.AspNetCore.Builder;
using Microsoft.AspNetCore.Http;
using Newtonsoft.Json;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Net;
using System.Text;
using System.Threading.Tasks;

namespace BoxTicketApi.Test.Project.Test
{
    public class ErrorHandelingMiddlewareTests
    {
```

```
        [Fact]
        public async Task Invoke_WhenExceptionThrown_ReturnsInternalServerError()
        {
            var middleware = new ErrorHandlingMiddleware(next: (innerHttpContext) =>
            {
                throw new Exception("Test Exception");
            });

            var context = new DefaultHttpContext();
            context.Response.Body = new System.IO.MemoryStream();

            await middleware.Invoke(context);
            context.Response.Body.Seek(0, System.IO.SeekOrigin.Begin);

            Assert.Equal((int)HttpStatusCode.InternalServerError,
context.Response.StatusCode);
            Assert.Equal("application/json", context.Response.ContentType);

            var response = await new
StreamReader(context.Response.Body).ReadToEndAsync();
            var result = JsonConvert.DeserializeObject<dynamic>(response);

            Assert.Equal("Test Exception", result.error.Value);
        }

        [Fact]
        public async Task Invoke_WhenNoExceptionThrown_PassesToNextMiddleware()
        {
            bool nextMiddlewareCalled = false;
            var middleware = new ErrorHandlingMiddleware(next: (innerHttpContext) =>
            {
                nextMiddlewareCalled = true;
                return Task.CompletedTask;
            });
            var context = new DefaultHttpContext();

            await middleware.Invoke(context);

            Assert.True(nextMiddlewareCalled);
        }


    }
}
using BoxTicketApi.BLL.Requests.Auth;
using BoxTicketApi.BLL.Requests.Genre;
using BoxTicketApi.BLL.Responses.Auth;
using BoxTicketApi.BLL.Responses.Genre;
using BoxTicketApi.BLL.Services.Base;
using BoxTicketApi.Controllers;
using BoxTicketApi.DAL.Entities;
using Microsoft.AspNetCore.Mvc;
using Moq;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace BoxTicketApi.Test.Project.Test
{
    public class GenreControllerTests
    {
        private readonly Mock<IGenreService> _genreServiceMock = new();
        private GenreController _genreController = null!;

        [Fact]
        public async Task GetAllGenre_ReturnGenreResponse()
        {
            List<GenreResponse> genre = new()
```

```
            {
                new()
                {
                    NameGenre="genre"
                }
            };

            _genreServiceMock.Setup(ds => ds.GetAllGenre())
                .ReturnsAsync(genre);
            _genreController = new GenreController(_genreServiceMock.Object);

            var returnedResult = await _genreController.GetAllGenre();

            var Result = Assert.IsType<OkObjectResult>(returnedResult);
            Assert.NotNull(Result.Value);
        }

        [Fact]
        public async Task AddGenre_ReturnGenreResponse()
        {
            GenreRequest request = new()
            {
                NameGenre = "genre"
            };

            GenreResponse genre = new()
            {
                Id = 1,
                NameGenre="genre"
            };

            _genreServiceMock.Setup(ds => ds.AddGenre(request))
                .ReturnsAsync(genre);
            _genreController = new GenreController(_genreServiceMock.Object);

            var returnedResult = await _genreController.AddGenre(request);

            var Result = Assert.IsType<OkObjectResult>(returnedResult);
            Assert.NotNull(Result.Value);
        }
    }
}
using BoxTicketApi.BLL.Requests.Author;
using BoxTicketApi.BLL.Requests.Performance;
using BoxTicketApi.BLL.Responses.Author;
using BoxTicketApi.BLL.Responses.Performance;
using BoxTicketApi.BLL.Services.Base;
using BoxTicketApi.Controllers;
using BoxTicketApi.DAL.Entities;
using Microsoft.AspNetCore.Mvc;
using Moq;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace BoxTicketApi.Test.Project.Test
{
    public class PerformanceControllerTests
    {
        private readonly Mock<IPerformanceService> _performanceServiceMock = new();
        private PerformanceController _performanceController = null!;

        [Fact]
        public async Task AddPerformance_ReturnPerformanceResponse()
        {
            PerformanceRequest request = new()
            {
                IdAuthor=1,
```

```
                IdGenre=1,
                DateTimeEvent=DateTime.Now,
                PerformanceName="Performance"
            };

            var response = new PerformanceResponse()
            {
                IdPerformance=1,
                IdAuthor = 1,
                IdGenre = 1,
                DateTimeEvent = DateTime.Now,
                PerformanceName = "Performance"
            };


            _performanceServiceMock.Setup(ds => ds.AddPerformance(request))
                .ReturnsAsync(response);
            _performanceController = new
PerformanceController(_performanceServiceMock.Object);

            var returnedResult = await
_performanceController.AddPerformances(request);

            var Result = Assert.IsType<OkObjectResult>(returnedResult);
            Assert.NotNull(Result.Value);
        }

        [Fact]
        public async Task GetAllPerformances_ReturnPerformancesResponse()
        {

            var response = new List<PerformanceResponse>()
            {
                new()
                {
                    IdPerformance = 1,
                    IdAuthor = 1,
                    IdGenre = 1,
                    DateTimeEvent = DateTime.Now,
                    PerformanceName = "Performance"
                }
            };

            _performanceServiceMock.Setup(ds => ds.GetAllPerformances())
                .ReturnsAsync(response);
            _performanceController = new
PerformanceController(_performanceServiceMock.Object);

            var returnedResult = await _performanceController.GetAllPerformances();

            var Result = Assert.IsType<OkObjectResult>(returnedResult);
            Assert.NotNull(Result.Value);
        }

        [Fact]
        public async Task GetPerformancesByDate_ReturnPerformanceResponse()
        {
            string date = "2023-03-04";
            PerformancesByDateRequest request = new()
            {
                Date = date,
                dateTime = new DateTime(2023, 03, 04)
            };

            var response = new List<PerformanceResponse>()
            {
                new()
                {
                    IdPerformance = 1,
                    IdAuthor = 1,
```

```
                    IdGenre = 1,
                    DateTimeEvent = request.dateTime,
                    PerformanceName = "Performance"
                }
            };

            _performanceServiceMock.Setup(ds =>
ds.GetPerformancesByDate(It.IsAny<PerformancesByDateRequest>()))
                .ReturnsAsync(response);
            _performanceController = new
PerformanceController(_performanceServiceMock.Object);

            var returnedResult = await
_performanceController.GetPerformancesByDate(date);

            var Result = Assert.IsType<OkObjectResult>(returnedResult);
            Assert.NotNull(Result.Value);
        }

        [Fact]
        public async Task GetPerformancesByAuthor_ReturnPerformanceResponse()
        {
            PerformancesByAuthorRequest request = new()
            {
                idAuthor =1
            };

            var response = new List<PerformanceResponse>()
            {
                new()
                {
                    IdPerformance = 1,
                    IdAuthor = 1,
                    IdGenre = 1,
                    DateTimeEvent = DateTime.Now,
                    PerformanceName = "Performance"
                }
            };

            _performanceServiceMock.Setup(ds => ds.GetPerformancesByAuthor(request))
                .ReturnsAsync(response);
            _performanceController = new
PerformanceController(_performanceServiceMock.Object);

            var returnedResult = await
_performanceController.GetPerformancesByAuthor(request);

            var Result = Assert.IsType<OkObjectResult>(returnedResult);
            Assert.NotNull(Result.Value);
        }

        [Fact]
        public async Task GetPerformancesByGenre_ReturnPerformanceResponse()
        {
            PerformancesByGenreRequest request = new()
            {
                idGenre = 1
            };

            var response = new List<PerformanceResponse>()
            {
                new()
                {
                    IdPerformance = 1,
                    IdAuthor = 1,
                    IdGenre = 1,
                    DateTimeEvent = DateTime.Now,
                    PerformanceName = "Performance"
                }
            };
```

```
            _performanceServiceMock.Setup(ds => ds.GetPerformancesByGenre(request))
                .ReturnsAsync(response);
            _performanceController = new
PerformanceController(_performanceServiceMock.Object);

            var returnedResult = await
_performanceController.GetPerformancesByGenre(request);

            var Result = Assert.IsType<OkObjectResult>(returnedResult);
            Assert.NotNull(Result.Value);
        }

        [Fact]
        public async Task GetPerformancesByName_ReturnPerformanceResponse()
        {
            PerformancesByNameRequest request = new()
            {
                Name = "Performance"
            };

            var response = new List<PerformanceResponse>()
            {
                new()
                {
                    IdPerformance = 1,
                    IdAuthor = 1,
                    IdGenre = 1,
                    DateTimeEvent = DateTime.Now,
                    PerformanceName = "Performance"
                }
            };

            _performanceServiceMock.Setup(ds => ds.GetPerformancesByName(request))
                .ReturnsAsync(response);
            _performanceController = new
PerformanceController(_performanceServiceMock.Object);

            var returnedResult = await
_performanceController.GetPerformancesByName(request);

            var Result = Assert.IsType<OkObjectResult>(returnedResult);
            Assert.NotNull(Result.Value);
        }
    }
}
using BoxTicketApi.BLL.Requests.Author;
using BoxTicketApi.BLL.Requests.Ticket;
using BoxTicketApi.BLL.Responses.Author;
using BoxTicketApi.BLL.Responses.Ticket;
using BoxTicketApi.BLL.Services;
using BoxTicketApi.BLL.Services.Base;
using BoxTicketApi.Controllers;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Moq;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace BoxTicketApi.Test.Project.Test
{
    public class TicketControllerTests
    {
        private readonly Mock<ITicketService> _ticketServiceMock = new();
        private readonly Mock<ITicketOptionsService> _ticketOptionService = new();
        private TicketController _ticketController = null!;
```

```
        [Fact]
        public async Task BookTicket_ReturnTicketResponse()
        {
            var request = new TicketReqest()
            {
                IdTicketOptions=1,
                IdUser=1,
                SeatNumber=3
            };
            TicketIdResponse response = new()
            {
                idTicket = 1
            };

            _ticketOptionService.Setup(ds =>
ds.GetIdPerformanceInOption(request.IdTicketOptions))
                .ReturnsAsync(1);
            _ticketServiceMock.Setup(ds => ds.BookTicket(request, 1))
                .ReturnsAsync(response);
            _ticketController = new TicketController(_ticketServiceMock.Object,
_ticketOptionService.Object);
            var httpContext = new DefaultHttpContext();
            httpContext.Request.Headers["Cookie"] = $"UserId={request.IdUser};
UserId={request.IdUser}";
            _ticketController.ControllerContext = new ControllerContext
            {
                HttpContext = httpContext
            };

            var returnedResult = await _ticketController.BookTicket(request);

            var Result = Assert.IsType<OkObjectResult>(returnedResult);
            Assert.NotNull(Result.Value);
        }

        [Fact]
        public async Task BookTicket_ReturnExeption()
        {
            var request = new TicketReqest()
            {
                IdTicketOptions = 1,
                IdUser = 1,
                SeatNumber = 3
            };
            TicketIdResponse response = new()
            {
                idTicket = 1
            };

            _ticketOptionService.Setup(ds =>
ds.GetIdPerformanceInOption(request.IdTicketOptions))
                .ReturnsAsync(0);
            _ticketServiceMock.Setup(ds => ds.BookTicket(request, 1))
                .ReturnsAsync(response);
            _ticketController = new TicketController(_ticketServiceMock.Object,
_ticketOptionService.Object);
            var httpContext = new DefaultHttpContext();
            httpContext.Request.Headers["Cookie"] = $"UserId={request.IdUser};
UserId={request.IdUser}";
            _ticketController.ControllerContext = new ControllerContext
            {
                HttpContext = httpContext
            };

            var exception = await Assert.ThrowsAsync<Exception>(async () => await
_ticketController.BookTicket(request));

            Assert.Equal($"Tcket with TicketOption id={request.IdTicketOptions} not
found.", exception.Message);
        }
```

```
        [Fact]
        public async Task BuyTicket_ReturnTicketResponse()
        {
            var request = new TicketReqest()
            {
                IdTicketOptions = 1,
                IdUser = 1,
                SeatNumber = 3
            };
            TicketIdResponse response = new()
            {
                idTicket = 1
            };

            _ticketOptionService.Setup(ds =>
ds.GetIdPerformanceInOption(request.IdTicketOptions))
                .ReturnsAsync(1);
            _ticketServiceMock.Setup(ds => ds.BuyTicket(request, 1))
                .ReturnsAsync(response);
            _ticketController = new TicketController(_ticketServiceMock.Object,
_ticketOptionService.Object);
            var httpContext = new DefaultHttpContext();
            httpContext.Request.Headers["Cookie"] = $"UserId={request.IdUser};
UserId={request.IdUser}";
            _ticketController.ControllerContext = new ControllerContext
            {
                HttpContext = httpContext
            };

            var returnedResult = await _ticketController.BuyTicket(request);

            var Result = Assert.IsType<OkObjectResult>(returnedResult);
            Assert.NotNull(Result.Value);
        }

        [Fact]
        public async Task BuyTicket_ReturnExeption()
        {
            var request = new TicketReqest()
            {
                IdTicketOptions = 1,
                IdUser = 1,
                SeatNumber = 3
            };
            TicketIdResponse response = new()
            {
                idTicket = 1
            };

            _ticketOptionService.Setup(ds =>
ds.GetIdPerformanceInOption(request.IdTicketOptions))
                .ReturnsAsync(0);
            _ticketServiceMock.Setup(ds => ds.BuyTicket(request, 1))
                .ReturnsAsync(response);
            _ticketController = new TicketController(_ticketServiceMock.Object,
_ticketOptionService.Object);
            var httpContext = new DefaultHttpContext();
            httpContext.Request.Headers["Cookie"] = $"UserId={request.IdUser};
UserId={request.IdUser}";
            _ticketController.ControllerContext = new ControllerContext
            {
                HttpContext = httpContext
            };

            var exception = await Assert.ThrowsAsync<Exception>(async () => await
_ticketController.BuyTicket(request));

            Assert.Equal($"Ticket with TicketOption id={request.IdTicketOptions} not
found.", exception.Message);
```

```
        }

        [Fact]
        public async Task BuyBookedTicket_ReturnTicketResponse()
        {
            var request = new TicketByIdReqest()
            {
                Id=1,
                IdUser=1
            };
            TicketIdResponse response = new()
            {
                idTicket = 1
            };

            _ticketServiceMock.Setup(ds => ds.BuyBookedTicket(request))
                .ReturnsAsync(response);
            _ticketController = new TicketController(_ticketServiceMock.Object,
_ticketOptionService.Object);
            var httpContext = new DefaultHttpContext();
            httpContext.Request.Headers["Cookie"] = $"UserId={request.IdUser};
UserId={request.IdUser}";
            _ticketController.ControllerContext = new ControllerContext
            {
                HttpContext = httpContext
            };

            var returnedResult = await _ticketController.BuyBookedTicket(request);

            var Result = Assert.IsType<OkObjectResult>(returnedResult);
            Assert.NotNull(Result.Value);
        }

        [Fact]
        public async Task GetMyTickets_ReturnTicketsResponse()
        {
            List<TicketResponse> response = new()
            {
                new()
                {
                    Id = 1,
                    IdPerformance = 1
                }
            };
            int userId = 1;

            _ticketServiceMock.Setup(ds => ds.GetAllTickets(userId))
                .ReturnsAsync(response);
            _ticketController = new TicketController(_ticketServiceMock.Object,
_ticketOptionService.Object);
            var httpContext = new DefaultHttpContext();
            httpContext.Request.Headers["Cookie"] = $"UserId={userId};
UserId={userId}";
            _ticketController.ControllerContext = new ControllerContext
            {
                HttpContext = httpContext
            };

            var returnedResult = await _ticketController.GetMyTickets();

            var Result = Assert.IsType<OkObjectResult>(returnedResult);
            Assert.NotNull(Result.Value);
        }

    }
}
using BoxTicketApi.BLL.Requests.Auth;
using BoxTicketApi.BLL.Requests.TicketOptions;
using BoxTicketApi.BLL.Responses.Auth;
using BoxTicketApi.BLL.Responses.TicketOptions;
```

```
using BoxTicketApi.BLL.Services.Base;
using BoxTicketApi.Controllers;
using Microsoft.AspNetCore.Mvc;
using Moq;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace BoxTicketApi.Test.Project.Test
{
    public class TicketOptionControllerTests
    {
        private readonly Mock<ITicketOptionsService> _ticketOptionServiceMock =
new();
        private TicketOptionsController _ticketOptionController = null!;

        [Fact]
        public async Task AvaillableTickets_ReturnOptionResponse()
        {
            var request = new GetOptionsRequest()
            {
                IdPerformance=1
            };
            var response = new List<OptionsResponse>()
            {
                new()
                {
                    CoutOfTickets=30,
                    NamePerformance="performance",
                    Id=1
                }
            };

            _ticketOptionServiceMock.Setup(ds => ds.GetAllAvailableTickets(request))
                .ReturnsAsync(response);
            _ticketOptionController = new
TicketOptionsController(_ticketOptionServiceMock.Object);

            var returnedResult = await
_ticketOptionController.GetAvaillableTickets(request);

            var Result = Assert.IsType<OkObjectResult>(returnedResult);
            Assert.NotNull(Result.Value);
        }
    }
}
```